

Coding Agent Prompt — Shopkeeper Live QR Screen

MyOffers Hyperlocal Platform

CONTEXT

You are building the **Shopkeeper Live QR Screen** for MyOffers — a hyperlocal offer platform where shopkeepers create offers and users claim them by visiting the shop physically. The QR screen is the shopkeeper's only in-session responsibility. It must be open on their phone at the counter so users can scan it to claim offers.

This is a **phone-first** feature. The shopkeeper has the same app as users but with a merchant role/dashboard.

SCOPE OF THIS TASK

Build the following:

1. **Backend** — QR token generation + validation API
2. **Frontend** — Shopkeeper's live QR screen (mobile UI)
3. **Token rotation logic** — auto-refresh every 30 seconds

Do NOT build: offer creation, user scanning flow, reward/coins logic, or auth system. Those are separate modules.

TECH STACK

- **Backend:** Node.js + Express (or NestJS if preferred)
- **Database:** PostgreSQL (offers, shops) + Redis (live QR tokens)
- **Frontend:** React Native (Expo) OR React (web, mobile viewport)
- **QR Library:** `react-native-qrcode-svg` (RN) or `qrcode.react` (web)
- **Encryption:** `crypto` (Node built-in), HMAC-SHA256

State your choice of RN vs React web at the top of your implementation.

BACKEND — QR TOKEN SYSTEM

1. Data Models (PostgreSQL)

```
sql

-- Already exists, reference only
shops (
  id UUID PRIMARY KEY,
  owner_user_id UUID,
  name TEXT,
  is_active BOOLEAN
)

offers (
  id UUID PRIMARY KEY,
  shop_id UUID REFERENCES shops(id),
  title TEXT,
  coins_for_user INTEGER,
  coins_for_shop INTEGER,
  valid_from TIMESTAMP,
  valid_until TIMESTAMP,
  is_active BOOLEAN
)

-- New table you must create
qr_token_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  token_hash TEXT NOT NULL,          -- SHA256 of raw token, for audit
  shop_id UUID NOT NULL,
  offer_id UUID NOT NULL,
  generated_at TIMESTAMP DEFAULT NOW(),
  expires_at TIMESTAMP NOT NULL,
  used BOOLEAN DEFAULT FALSE,
  used_at TIMESTAMP
)
```

2. Redis Key Structure

```
qr:live:{shop_id}:{offer_id} → { token, expires_at, generation }
TTL: 35 seconds (5s buffer over 30s display window)

qr:used:{token_hash} → "1"
TTL: 120 seconds (prevent replay within cool-down)
```

3. API Endpoints

POST /api/shopkeeper/qr/generate

Purpose: Generate a fresh QR token for a specific offer. Called by the shopkeeper's app every 30 seconds via polling or called once and rotated server-side.

Auth: Bearer token (shopkeeper JWT). Verify `shop.owner_user_id === req.user.id`.

Request body:

```
json
{
  "offer_id": "uuid"
}
```

Token construction (do exactly this):

```
raw_token = base64url(
  JSON.stringify({
    shop_id,
    offer_id,
    ts: unix_timestamp_seconds,
    nonce: crypto.randomBytes(8).hex()
  })
)

signature = HMAC-SHA256(raw_token, QR_SECRET_KEY) // from env

final_token = raw_token + "." + signature
```

`QR_SECRET_KEY` must be in `.env`. Never expose it to frontend.

Response:

```
json
{
  "token": "eyJ...", // full signed token – embed this in QR
  "expires_at": 1712345678, // unix timestamp
  "offer_id": "uuid",
  "offer_title": "20% off on all items",
  "coins_preview": 50, // coins user will receive
  "generated_at": 1712345648
}
```

Side effects:

- Store token_hash + metadata in `qr_token_log`
- Set Redis key `qr:live:{shop_id}:{offer_id}` with 35s TTL

Error cases:

- 403 if shopkeeper doesn't own this shop
 - 404 if offer not found or not active
 - 400 if offer validity window has passed
-

GET `/api/shopkeeper/qr/current?offer_id={uuid}`

Purpose: Return current live token without generating a new one. Used on screen resume/focus restore.

Logic:

- Check Redis for `qr:live:{shop_id}:{offer_id}`
- If exists and TTL > 5s → return it
- If missing or TTL ≤ 5s → auto-call generate logic and return fresh token

Response: Same shape as generate endpoint.

POST `/api/qr/validate` (*called by USER app, not shopkeeper*)

Purpose: Validate a scanned QR token. This is the claim gate.

Request body:

```
json
{
  "token": "eyJ...",
  "user_id": "uuid"
}
```

Validation steps (all must pass, in this order):

1. **Parse token** — split on `(.)`, decode base64url payload, verify HMAC signature
2. **Timestamp check** — `payload.ts` must be within last 35 seconds
3. **Redis used-check** — `qr:used:{token_hash}` must NOT exist
4. **Offer active check** — offer must be `is_active=true` and within `valid_from`/`valid_until`

5. **Unlock check** — user must have an active unlock record for this offer (check PostgreSQL `offer_unlocks` table)
6. **Session check** — user must have an active visit session (check Redis `session:{user_id}:{offer_id}`)
7. **Duplicate claim check** — no existing `claims` record with `user_id + offer_id + status` IN (`'pending'`, `'claimed'`)

On all checks passing:

- Set `qr:used:{token_hash}` in Redis with 120s TTL (burn the token)
- Insert into `claims` table with `status = 'pending'`
- Publish event to queue: `{ type: 'CLAIM_PENDING', user_id, offer_id, shop_id }`
- Return:

```
json
{
  "success": true,
  "claim_id": "uuid",
  "message": "Visit confirmed! Coins will be credited shortly.",
  "coins_pending": 50
}
```

On any check failing, return specific error:

```
json
{
  "success": false,
  "error": "TOKEN_EXPIRED" | "ALREADY_USED" | "NO_UNLOCK" | "NO_SESSION" | "ALREADY_"
}
```

4. Environment Variables Required

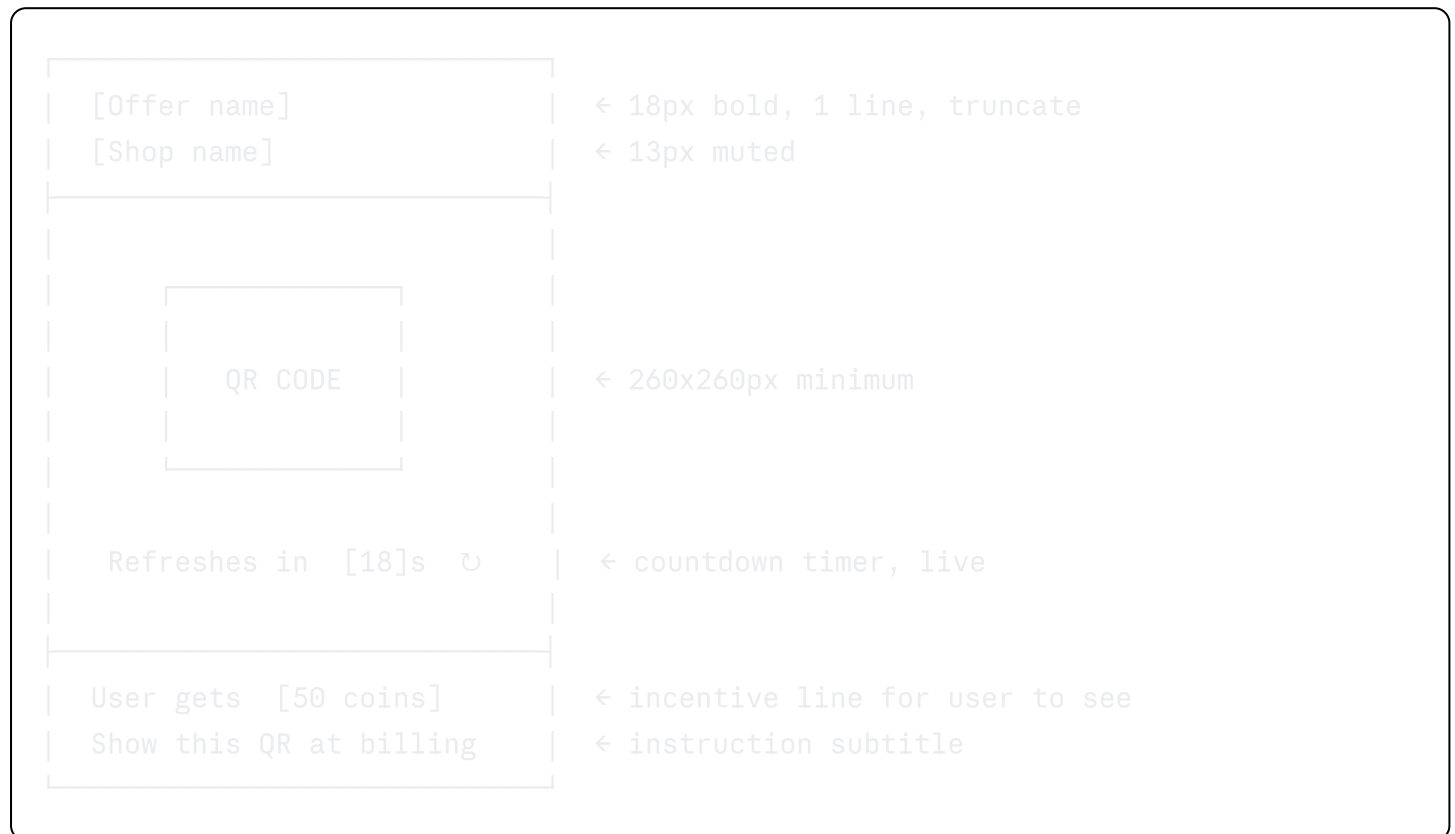
```
env
QR_SECRET_KEY=<random 64-char hex string>
REDIS_URL=redis://localhost:6379
DATABASE_URL=postgresql://...
JWT_SECRET=...
```

FRONTEND — SHOPKEEPER QR SCREEN

Screen: `ShopkeeperQRScreen`

This is a **full-screen, always-on display**. The shopkeeper opens this and shows it to customers. It must be readable from 30–50cm distance.

Layout (top to bottom)



Component Behaviour

On mount:

1. Call `GET /api/shopkeeper/qr/current?offer_id={id}` to load initial token
2. Render QR immediately
3. Start 30-second countdown timer
4. At T=0, call `POST /api/shopkeeper/qr/generate` to get new token
5. Swap QR with brief fade transition (150ms opacity 0→1)
6. Restart countdown

Keep screen alive:

- Call `navigator.wakeLock.request('screen')` on mount (web) or use `expo-keep-awake` (RN) — screen must NOT sleep while this view is open

- Release wake lock on unmount

On app background/foreground:

- On foreground resume: call `GET /api/shopkeeper/qr/current` — if token has < 5s left, force-regenerate immediately

Loading state: Show skeleton/spinner only on very first load. Subsequent refreshes are silent — QR swaps without full reload.

Error state: If API fails, show retry button. Do not show a broken/empty QR — it confuses users at the counter.

State Shape

typescript

```
interface QRScreenState {
  token: string | null;
  expiresAt: number | null;      // unix timestamp
  secondsLeft: number;          // drives countdown UI
  offerTitle: string;
  shopName: string;
  coinsForUser: number;
  status: 'loading' | 'live' | 'refreshing' | 'error';
}
```

Visual Design Requirements

- **Background:** Dark (near-black `#0F0F0F` or deep navy). High contrast.
- **QR code:** White modules on white card background — maximum scan reliability
- **QR card:** White rounded card (`border-radius: 16px`), generous padding (24px), subtle shadow
- **Countdown:** Color shifts from green → amber → red as it approaches 0
 - 15s = green (`#22C55E`)
 - 6-15s = amber (`#F59E0B`)
 - ≤ 5s = red (`#EF4444`) + pulse animation
- **Coins badge:** Prominent pill badge showing coin amount — users look for this
- **Font:** System font or a clean sans — readability over style here
- **No navigation bar** on this screen — full screen, distraction-free

Countdown Timer Logic

typescript

```
useEffect(() => {
  const interval = setInterval(() => {
    const secondsLeft = Math.max(0, expiresAt - Math.floor(Date.now() / 1000));
    setSecondsLeft(secondsLeft);

    if (secondsLeft <= 1) {
      refreshToken(); // call generate endpoint
    }
  }, 1000);

  return () => clearInterval(interval);
}, [expiresAt]);
```

Offer Selector (above QR screen)

The shopkeeper may have multiple active offers. Show a **horizontal pill selector** above the QR (or a bottom sheet picker) listing active offers by name. Tapping one reloads the QR for that offer. Default to the first active offer on load.

typescript

```
// Fetch active offers for this shop on screen mount
GET /api/shopkeeper/offers/active
→ [{ id, title, coins_for_user, valid_until }]
```

FOLDER STRUCTURE

```
/backend
  /src
    /qr
      qr.controller.ts    ← 3 endpoints above
      qr.service.ts       ← token gen, validation, redis ops
      qr.utils.ts         ← HMAC sign/verify, token encode/decode
    /middleware
      shopkeeper.guard.ts ← verifies merchant role

/frontend
```



```
/screens
  ShopkeeperQRScreen.tsx ← main screen
/components
  LiveQRCard.tsx          ← QR + countdown + coins badge
  OfferPillSelector.tsx   ← offer switcher
/hooks
  useQRToken.ts           ← polling, rotation, wake lock logic
/api
  qr.api.ts               ← API call wrappers
```

SECURITY RULES — DO NOT SKIP

1. `QR_SECRET_KEY` never leaves the backend. Never send it to frontend.
 2. Always verify HMAC signature before trusting any token payload.
 3. Token burn (`qr:used`) must happen BEFORE inserting the claim — not after.
 4. All QR validate calls must be idempotent — same token scanned twice returns `ALREADY_USED`, never creates two claims.
 5. Shopkeeper ownership check is mandatory on every generate call — never let one shopkeeper generate QR for another shop.
-

DELIVERABLES EXPECTED

- ☐ `qr.service.ts` (or `qr.service.js`) — full token generation + validation logic
 - ☐ `qr.controller.ts` — all 3 endpoints with error handling
 - ☐ `ShopkeeperQRScreen.tsx` — complete screen component
 - ☐ `useQRToken.ts` — custom hook with rotation + wake lock
 - ☐ `LiveQRCard.tsx` — QR display with countdown
 - ☐ Redis + PostgreSQL setup commands / migration SQL
 - ☐ `.env.example` with all required keys
-

WHAT SUCCESS LOOKS LIKE

- Shopkeeper opens app → taps "My QR" → full-screen QR appears in < 1 second
- QR refreshes every 30 seconds silently, counter resets
- Screen never sleeps while QR is displayed
- A user scanning the QR within its 30s window receives a valid token
- Same token scanned twice returns `ALREADY_USED`

- A screenshot of the QR is invalid after 35 seconds maximum